

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284513

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Horwitz; Joshua et al.

Virtual Agent For Container Orchestration System

Abstract

Techniques for a container orchestration system are disclosed. A virtual agent may be executed in a first service tenancy of a cloud network manager. The virtual agent may be executed on a virtual node of a container orchestration system. A container orchestration API server may be executed in a second service tenancy of the cloud network manager. A request from the container orchestration API server to the virtual agent may be routed through a customer tenancy of a customer of the cloud network manager.

Inventors: Horwitz; Joshua (Centreville, VA), Puranik; Srinidhi Chokkadi (San Jose, CA), Curtis; Matthew Raymond (Melbourne, AU), Bharadwaj; Rtvik Sriram (Seattle, WA)

Applicant: Oracle International Corporation (Redwood Shores, CA)

Family ID: 94734133

Assignee: Oracle International Corporation (Redwood Shores, CA)

Appl. No.: 18/595955

Filed: March 05, 2024

Publication Classification

Int. Cl.: G06F9/455 (20180101); H04L67/10 (20220101)

U.S. Cl.:

CPC G06F9/45558 (20130101); H04L67/10 (20130101); G06F2009/45562 (20130101); G06F2009/4557 (20130101); G06F2009/45595 (20130101)

Background/Summary

TECHNICAL FIELD

[0001] The present disclosure relates to container orchestration systems. In particular, the present disclosure relates to virtual agents for container orchestration systems.

BACKGROUND

[0002] Container orchestration is the process of automating the deployment, scaling, and management of containerized applications. Containers allow developers to package an application and its dependencies into a single unit to ensure consistency across different environments. Container orchestration involves automating the provisioning, deployment, networking, scaling, availability, and lifecycle management of containers. Container orchestration helps to simplify the process of deploying and managing containers, especially when dealing with large-scale applications. Kubernetes is currently the most popular container orchestration platform and is widely used by leading public cloud providers.

[0003] The approaches described in this section are approaches that could be pursued, but not necessarily approaches that have been previously conceived or pursued. Therefore, unless otherwise indicated, it should not be assumed that any of the approaches described in this section qualify as prior art merely by virtue of their inclusion in this section.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] The embodiments are illustrated by way of example and not by way of limitation in the figures of the accompanying drawings. It should be noted that references to “an” or “one” embodiment in this disclosure are not necessarily to the same embodiment, and they mean at least one. In the drawings:

[0005] FIGS. 1-4 are block diagrams illustrating patterns for implementing a cloud infrastructure as a service system in accordance with one or more embodiments;

[0006] FIG. 5 is a hardware system in accordance with one or more embodiments;

[0007] FIG. 6 illustrates a system in accordance with one or more embodiments;

[0008] FIG. 7 illustrates an example set of operations for preparing a virtual agent for container orchestration in accordance with one or more embodiments;

[0009] FIG. 8 illustrates an example set of operations for container orchestration using a virtual agent in a cloud environment in accordance with one or more embodiments.

DETAILED DESCRIPTION

[0010] In the following description, for the purposes of explanation, numerous specific details are set forth to provide a thorough understanding. One or more embodiments may be practiced without these specific details. Features described in one embodiment may be combined with features described in a different embodiment. In some examples, well-known structures and devices are described with reference to a block diagram form to avoid unnecessarily obscuring the present disclosure. [0011] 1. GENERAL OVERVIEW [0012] 2. CLOUD COMPUTING TECHNOLOGY [0013] 3. COMPUTER SYSTEM [0014] 4. CONTAINER ORCHESTRATION SYSTEM [0015] 5. CONTAINER ORCHESTRATION METHOD [0016] 6. CONTAINER ORCHESTRATION SIGNALING [0017] 7. MISCELLANEOUS; EXTENSIONS

1. GENERAL OVERVIEW

[0018] One or more embodiments route a request between the container orchestration API server and the virtual agent, executing on different service tenancies of a cloud network, through a customer tenancy of a customer of the cloud network manager. The virtual agent executes on a virtual node of a container orchestration system. Alternatively, or additionally, the virtual agent executes as a virtual kubelet used in a Kubernetes cluster and/or another container orchestration

system. The container orchestration API server is a Kubernetes API server used in a Kubernetes cluster or is another type of container orchestration API server used in another container orchestration system.

[0019] Implementing the container orchestration API server and virtual agent within the service tenancies is conducive to the cloud network manager maintaining and updating the container orchestration API server and the virtual agent. The system patches a version of the virtual agent in the first service tenancy under the control of the cloud network without requiring a request from the customer. Further, the system protects the virtual agent in the first service tenancy using cloud network security for the first service tenancy.

[0020] One or more embodiments implement at least one of the service tenancies and the customer tenancy within the same cloud environment configured to execute operations corresponding to a data set associated with the customer. In one example, the cloud environment implements a container orchestration cluster, such as a Kubernetes cluster. The system implements the container orchestration cluster and a customer tenancy within the same cloud environment.

[0021] One or more embodiments include a first virtual agent replica executed by a first host and a second virtual agent replica executed by a second host as the virtual agent. The second virtual agent replica operates even when the first virtual agent replica fails.

[0022] In an embodiment, routing the request through the customer tenancy includes addressing the request to a service virtual network interface controller (VNIC) in the customer tenancy that sends the request to the virtual agent. Since the virtual agent is able to control multiple containers in multiple container instances, the number of containers of the virtual node is greater than in a conventional node of a container orchestration cluster.

[0023] One or more embodiments described in this Specification and/or recited in the claims may not be included in this General Overview section.

2. CLOUD COMPUTING TECHNOLOGY

[0024] Infrastructure as a Service (IaaS) is an application of cloud computing technology. IaaS can be configured to provide virtualized computing resources over a public network (e.g., the Internet). In an IaaS model, a cloud computing provider can host the infrastructure components (e.g., servers, storage devices, network nodes (e.g., hardware), deployment software, platform virtualization (e.g., a hypervisor layer), or the like). In some cases, an IaaS provider may also supply a variety of services to accompany those infrastructure components (example services include billing software, monitoring software, logging software, load balancing software, clustering software, etc.). Thus, as these services may be policy-driven, IaaS users may be able to implement policies to drive load balancing to maintain application availability and performance.

[0025] In some instances, IaaS customers may access resources and services through a wide area network (WAN), such as the Internet, and can use the cloud provider's services to install the remaining elements of an application stack. For example, the user can log in to the IaaS platform to create virtual machines (VMs), install operating systems (OSs) on each VM, deploy middleware such as databases, create storage buckets for workloads and backups, and even install enterprise software into that VM. Customers can then use the provider's services to perform various functions, including balancing network traffic, troubleshooting application issues, monitoring performance, managing disaster recovery, etc.

[0026] In some cases, a cloud computing model will involve the participation of a cloud provider. The cloud provider may, but need not, be a third-party service that specializes in providing (e.g., offering, renting, selling) IaaS. An entity may also opt to deploy a private cloud, becoming its own provider of infrastructure services.

[0027] In some examples, IaaS deployment is the process of implementing a new application, or a new version of an application, onto a prepared application server or other similar device. IaaS deployment may also include the process of preparing the server (e.g., installing libraries, daemons, etc.). The deployment process is often managed by the cloud provider below the hypervisor layer

(e.g., the servers, storage, network hardware, and virtualization). Thus, the customer may be responsible for handling Operating System (OS), middleware, and/or application deployment e.g., on self-service virtual machines that can be spun up on demand.

[0028] In some examples, IaaS provisioning may refer to acquiring computers or virtual hosts for use and even installing needed libraries or services on them. In most cases, deployment does not include provisioning, and the provisioning may need to be performed first.

[0029] In some cases, there are challenges for IaaS provisioning. There is an initial challenge of provisioning the initial set of infrastructure. There is an additional challenge of evolving the existing infrastructure (e.g., adding new services, changing services, removing services, etc.) after the initial provisioning is completed. In some cases, these challenges may be addressed by enabling the configuration of the infrastructure to be defined declaratively. In other words, the infrastructure (e.g., what components are needed and how they interact) can be defined by one or more configuration files. Thus, the overall topology of the infrastructure (e.g., what resources depend on which, and how they each work together) can be described declaratively. In some instances, once the topology is defined, a workflow can be generated that creates and/or manages the different components described in the configuration files.

[0030] In some examples, an infrastructure may have many interconnected elements. For example, there may be one or more virtual private clouds (VPCs) (e.g., a potentially on-demand pool of configurable and/or shared computing resources), also known as a core network. In some examples, there may also be one or more inbound/outbound traffic group rules provisioned to define how the inbound and/or outbound traffic of the network will be set up. Other infrastructure elements may also be provisioned, such as a load balancer, a database, or the like. As more and more infrastructure elements are desired and/or added, the infrastructure may incrementally evolve.

[0031] In some instances, continuous deployment techniques may be employed to enable deployment of infrastructure code across various virtual computing environments. Additionally, the described techniques can enable infrastructure management within these environments. In some examples, service teams can write code that is desired to be deployed to one or more, but often many, different production environments (e.g., across various different geographic locations, sometimes spanning the entire world). In some embodiments, infrastructure and resources may be provisioned (manually, and/or using a provisioning tool) prior to deployment of code to be executed on the infrastructure. In some instances, the provisioning can be done manually, a provisioning tool may be utilized to provision the resources, and/or deployment tools may be utilized to deploy the code once the infrastructure is provisioned.

[0032] FIG. 1 is a block diagram illustrating an example pattern of an IaaS architecture **100** according to at least one embodiment. Service operators **102** can be communicatively coupled to a secure host tenancy **104** that can include a virtual cloud network (VCN) **106** and a secure host subnet **108**. In some examples, the service operators **102** may be using one or more client computing devices that may be portable handheld devices (e.g., an iPhone®, cellular telephone, an iPad®, computing tablet, a personal digital assistant (PDA)) or wearable devices (e.g., a Google Glass® head mounted display), running software such as Microsoft Windows Mobile®, and/or a variety of mobile operating systems such as iOS, Windows Phone, Android, BlackBerry 8, Palm OS, and the like, and being Internet, e-mail, short message service (SMS), Blackberry®, or other communication protocol enabled. Alternatively, the client computing devices can be general purpose personal computers including, by way of example, personal computers and/or laptop computers running various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems. The client computing devices can be workstation computers executing any of a variety of commercially-available UNIX® or UNIX-like operating systems, including without limitation the variety of GNU/Linux operating systems, such as for example, Google Chrome OS. Additionally, or alternatively, client computing devices may be any other electronic device, such as a thin-client computer, an Internet-enabled gaming system (e.g., a Microsoft Xbox gaming console

with or without a Kinect® gesture input device), and/or a personal messaging device, capable of communicating over a network that can access the VCN **106** and/or the Internet.

[0033] The VCN **106** can include a local peering gateway (LPG) **110** that can be communicatively coupled to a secure shell (SSH) VCN **112** via an LPG **110** contained in the SSH VCN **112**. The SSH VCN **112** can include an SSH subnet **114**, and the SSH VCN **112** can be communicatively coupled to a control plane VCN **116** via the LPG **110** contained in the control plane VCN **116**. Also, the SSH VCN **112** can be communicatively coupled to a data plane VCN **118** via an LPG **110**. The control plane VCN **116** and the data plane VCN **118** can be contained in a service tenancy **119** that can be owned and/or operated by the IaaS provider.

[0034] The control plane VCN **116** can include a control plane demilitarized zone (DMZ) tier **120** that acts as a perimeter network (e.g., portions of a corporate network between the corporate intranet and external networks). The DMZ-based servers may have restricted responsibilities and help keep breaches contained. Additionally, the DMZ tier **120** can include one or more load balancer (LB) subnet(s) **122**, a control plane app tier **124** that can include app subnet(s) **126**, a control plane data tier **128** that can include database (DB) subnet(s) **130** (e.g., frontend DB subnet(s) and/or backend DB subnet(s)). The LB subnet(s) **122** contained in the control plane DMZ tier **120** can be communicatively coupled to the app subnet(s) **126** contained in the control plane app tier **124**. The LB subnet(s) **122** may further be communicatively coupled to an Internet gateway **134** that can be contained in the control plane VCN **116**. The app subnet(s) **126** can be communicatively coupled to the DB subnet(s) **130** contained in the control plane data tier **128**, a service gateway **136** and a network address translation (NAT) gateway **138**. The control plane VCN **116** can include the service gateway **136** and the NAT gateway **138**.

[0035] The control plane VCN **116** can include a data plane mirror app tier **140** that can include app subnet(s) **126**. The app subnet(s) **126** contained in the data plane mirror app tier **140** can include a virtual network interface controller (VNIC) **142** that can execute a compute instance **144**. The compute instance **144** can communicatively couple the app subnet(s) **126** of the data plane mirror app tier **140** to app subnet(s) **126** that can be contained in a data plane app tier **146**.

[0036] The data plane VCN **118** can include the data plane app tier **146**, a data plane DMZ tier **148**, and a data plane data tier **150**. The data plane DMZ tier **148** can include LB subnet(s) **122** that can be communicatively coupled to the app subnet(s) **126** of the data plane app tier **146** and the Internet gateway **134** of the data plane VCN **118**. The app subnet(s) **126** can be communicatively coupled to the service gateway **136** of the data plane VCN **118** and the NAT gateway **138** of the data plane VCN **118**. The data plane data tier **150** can also include the DB subnet(s) **130** that can be communicatively coupled to the app subnet(s) **126** of the data plane app tier **146**.

[0037] The Internet gateway **134** of the control plane VCN **116** and of the data plane VCN **118** can be communicatively coupled to a metadata management service **152** that can be communicatively coupled to public Internet **154**. Public Internet **154** can be communicatively coupled to the NAT gateway **138** of the control plane VCN **116** and of the data plane VCN **118**. The service gateway **136** of the control plane VCN **116** and of the data plane VCN **118** can be communicatively couple to cloud services **156**.

[0038] In some examples, the service gateway **136** of the control plane VCN **116** or of the data plane VCN **118** can make application programming interface (API) calls to cloud services **156** without going through public Internet **154**. The service gateway **136** can make API calls to cloud services **156**, and cloud services **156** can send requested data to the service gateway **136**.

[0039] In some examples, the secure host tenancy **104** can be directly connected to the service tenancy **119** that may be otherwise isolated. The secure host subnet **108** can communicate with the SSH subnet **114** through an LPG **110** that may enable two-way communication over an otherwise isolated system. Connecting the secure host subnet **108** to the SSH subnet **114** may give the secure host subnet **108** access to other entities within the service tenancy **119**.

[0040] The control plane VCN **116** may allow users of the service tenancy **119** to set up or

otherwise provision desired resources. Desired resources provisioned in the control plane VCN **116** may be deployed or otherwise used in the data plane VCN **118**. In some examples, the control plane VCN **116** can be isolated from the data plane VCN **118**. The data plane mirror app tier **140** of the control plane VCN **116** can communicate with the data plane app tier **146** of the data plane VCN **118** via VNICs **142**. VNICs **142** can be contained in the data plane mirror app tier **140** and the data plane app tier **146**.

[0041] In some examples, users of the system, or customers, can make requests, for example create, read, update, or delete (CRUD) operations, through public Internet **154** that can communicate the requests to the metadata management service **152**. The metadata management service **152** can communicate the request to the control plane VCN **116** through the Internet gateway **134**. The request can be received by the LB subnet(s) **122** contained in the control plane DMZ tier **120**. The LB subnet(s) **122** may determine that the request is valid, and in response to this determination, the LB subnet(s) **122** can transmit the request to app subnet(s) **126** contained in the control plane app tier **124**. If the request is validated and requires a call to public Internet **154**, the call to public Internet **154** may be transmitted to the NAT gateway **138** that can make the call to public Internet **154**. Metadata that may be desired to be stored by the request can be stored in the DB subnet(s) **130**.

[0042] In some examples, the data plane mirror app tier **140** can facilitate direct communication between the control plane VCN **116** and the data plane VCN **118**. For example, changes, updates, or other suitable modifications to configuration may be desired to be applied to the resources contained in the data plane VCN **118**. Via a VNIC **142**, the control plane VCN **116** can directly communicate with, and can thereby execute the changes, updates, or other suitable modifications to configurations of resources contained in the data plane VCN **118**.

[0043] In some embodiments, the control plane VCN **116** and the data plane VCN **118** can be contained in the service tenancy **119**. The user, or the customer, of the system may be restricted from owning or operating either the control plane VCN **116** or the data plane VCN **118**. Instead, the IaaS provider may own or operate the control plane VCN **116** and the data plane VCN **118**, both of which may be contained in the service tenancy **119**. This embodiment can enable isolation of networks that may prevent users or customers from interacting with other users' or other customers' resources. Also, this embodiment may allow users or customers of the system to store databases privately without needing to rely on public Internet **154** that may not have a desired level of threat prevention for storage.

[0044] In other embodiments, the LB subnet(s) **122** contained in the control plane VCN **116** can be configured to receive a signal from the service gateway **136**. In this embodiment, the control plane VCN **116** and the data plane VCN **118** may be configured to be called by a customer of the IaaS provider without calling public Internet **154**. Customers of the IaaS provider may desire this embodiment since database(s) that the customers use may be controlled by the IaaS provider and may be stored on the service tenancy **119** that may be isolated from public Internet **154**.

[0045] FIG. 2 is a block diagram illustrating another example pattern of an IaaS architecture **200**, according to at least one embodiment. Service operators **202** (e.g., service operators **102** of FIG. 1) can be communicatively coupled to a secure host tenancy **204** (e.g., the secure host tenancy **104** of FIG. 1) that can include a virtual cloud network (VCN) **206** (e.g., the VCN **106** of FIG. 1) and a secure host subnet **208** (e.g., the secure host subnet **108** of FIG. 1). The VCN **206** can include a local peering gateway (LPG) **210** (e.g., the LPG **110** of FIG. 1) that can be communicatively coupled to a secure shell (SSH) VCN **212** (e.g., the SSH VCN **112** of FIG. 1) via an LPG **110** contained in the SSH VCN **212**. The SSH VCN **212** can include an SSH subnet **214** (e.g., the SSH subnet **114** of FIG. 1), and the SSH VCN **212** can be communicatively coupled to a control plane VCN **216** (e.g., the control plane VCN **116** of FIG. 1) via an LPG **210** contained in the control plane VCN **216**. The control plane VCN **216** can be contained in a service tenancy **219** (e.g., the service tenancy **119** of FIG. 1), and the data plane VCN **218** (e.g., the data plane VCN **118** of FIG.

1) can be contained in a customer tenancy **221** that may be owned or operated by users, or customers, of the system.

[0046] The control plane VCN **216** can include a control plane DMZ tier **220** (e.g., the control plane DMZ tier **120** of FIG. 1) that can include LB subnet(s) **222** (e.g., LB subnet(s) **122** of FIG. 1), a control plane app tier **224** (e.g., the control plane app tier **124** of FIG. 1) that can include app subnet(s) **226** (e.g., app subnet(s) **126** of FIG. 1), and a control plane data tier **228** (e.g., the control plane data tier **128** of FIG. 1) that can include database (DB) subnet(s) **230** (e.g., similar to DB subnet(s) **130** of FIG. 1). The LB subnet(s) **222** contained in the control plane DMZ tier **220** can be communicatively coupled to the app subnet(s) **226** contained in the control plane app tier **224** and an Internet gateway **234** (e.g., the Internet gateway **134** of FIG. 1). The Internet gateway **234** can be contained in the control plane VCN **216**. Additionally, the app subnet(s) **226** can be communicatively coupled to the DB subnet(s) **230** contained in the control plane data tier **228**, a service gateway **236** (e.g., the service gateway **136** of FIG. 1) and a network address translation (NAT) gateway **238** (e.g., the NAT gateway **138** of FIG. 1). The control plane VCN **216** can include the service gateway **236** and the NAT gateway **238**.

[0047] The control plane VCN **216** can include a data plane mirror app tier **240** (e.g., the data plane mirror app tier **140** of FIG. 1) that can include app subnet(s) **226**. The app subnet(s) **226** contained in the data plane mirror app tier **240** can include a virtual network interface controller (VNIC) **242** (e.g., the VNIC of **142**) that can execute a compute instance **244** (e.g., similar to the compute instance **144** of FIG. 1). The compute instance **244** can facilitate communication between the app subnet(s) **226** of the data plane mirror app tier **240** and the app subnet(s) **226** that can be contained in a data plane app tier **246** (e.g., the data plane app tier **146** of FIG. 1) via the VNIC **242** contained in the data plane mirror app tier **240** and the VNIC **242** contained in the data plane app tier **246**.

[0048] The Internet gateway **234** contained in the control plane VCN **216** can be communicatively coupled to a metadata management service **252** (e.g., the metadata management service **152** of FIG. 1) that can be communicatively coupled to public Internet **254** (e.g., public Internet **154** of FIG. 1). Public Internet **254** can be communicatively coupled to the NAT gateway **238** contained in the control plane VCN **216**. The service gateway **236** contained in the control plane VCN **216** can be communicatively couple to cloud services **256** (e.g., cloud services **156** of FIG. 1).

[0049] In some examples, the data plane VCN **218** can be contained in the customer tenancy **221**. In this case, the IaaS provider may provide the control plane VCN **216** for each customer, and the IaaS provider may, for each customer, set up a unique, compute instance **244** that is contained in the service tenancy **219**. Each compute instance **244** may allow communication between the control plane VCN **216**, contained in the service tenancy **219**, and the data plane VCN **218**, contained in the customer tenancy **221**. The compute instance **244** may allow resources provisioned in the control plane VCN **216** that is contained in the service tenancy **219** to be deployed or otherwise used in the data plane VCN **218** that is contained in the customer tenancy **221**.

[0050] In other examples, the customer of the IaaS provider may have databases that live in the customer tenancy **221**. In this example, the control plane VCN **216** can include the data plane mirror app tier **240** that can include app subnet(s) **226**. The data plane mirror app tier **240** can reside in the data plane VCN **218**, but the data plane mirror app tier **240** may not live in the data plane VCN **218**. That is, the data plane mirror app tier **240** may have access to the customer tenancy **221**, but the data plane mirror app tier **240** may not exist in the data plane VCN **218** or be owned or operated by the customer of the IaaS provider. The data plane mirror app tier **240** may be configured to make calls to the data plane VCN **218** but may not be configured to make calls to any entity contained in the control plane VCN **216**. The customer may desire to deploy or otherwise use resources in the data plane VCN **218** that are provisioned in the control plane VCN **216**, and the data plane mirror app tier **240** can facilitate the desired deployment, or other usage of resources, of the customer.

[0051] In some embodiments, the customer of the IaaS provider can apply filters to the data plane

VCN **218**. In this embodiment, the customer can determine what the data plane VCN **218** can access, and the customer may restrict access to public Internet **254** from the data plane VCN **218**. The IaaS provider may not be able to apply filters or otherwise control access of the data plane VCN **218** to any outside networks or databases. Applying filters and controls by the customer onto the data plane VCN **218**, contained in the customer tenancy **221**, can help isolate the data plane VCN **218** from other customers and from public Internet **254**.

[0052] In some embodiments, cloud services **256** can be called by the service gateway **236** to access services that may not exist on public Internet **254**, on the control plane VCN **216**, or on the data plane VCN **218**. The connection between cloud services **256** and the control plane VCN **216** or the data plane VCN **218** may not be live or continuous. Cloud services **256** may exist on a different network owned or operated by the IaaS provider. Cloud services **256** may be configured to receive calls from the service gateway **236** and may be configured to not receive calls from public Internet **254**. Some cloud services **256** may be isolated from other cloud services **256**, and the control plane VCN **216** may be isolated from cloud services **256** that may not be in the same region as the control plane VCN **216**. For example, the control plane VCN **216** may be located in Region **1**, and cloud service Deployment **1** may be located in Region **1** and in Region **2**. If a call to Deployment **1** is made by the service gateway **236** contained in the control plane VCN **216** located in Region **1**, the call may be transmitted to Deployment **1** in Region **1**. In this example, the control plane VCN **216**, or Deployment **1** in Region **1**, may not be communicatively coupled to, or otherwise in communication with, Deployment **1** in Region **2**.

[0053] FIG. **3** is a block diagram illustrating another example pattern of an IaaS architecture **300** according to at least one embodiment. Service operators **302** (e.g., service operators **102** of FIG. **1**) can be communicatively coupled to a secure host tenancy **304** (e.g., the secure host tenancy **104** of FIG. **1**) that can include a virtual cloud network (VCN) **306** (e.g., the VCN **106** of FIG. **1**) and a secure host subnet **308** (e.g., the secure host subnet **108** of FIG. **1**). The VCN **306** can include an LPG **310** (e.g., the LPG **110** of FIG. **1**) that can be communicatively coupled to an SSH VCN **312** (e.g., the SSH VCN **112** of FIG. **1**) via an LPG **310** contained in the SSH VCN **312**. The SSH VCN **312** can include an SSH subnet **314** (e.g., the SSH subnet **114** of FIG. **1**), and the SSH VCN **312** can be communicatively coupled to a control plane VCN **316** (e.g., the control plane VCN **116** of FIG. **1**) via an LPG **310** contained in the control plane VCN **316** and to a data plane VCN **318** (e.g., the data plane VCN **118** of FIG. **1**) via an LPG **310** contained in the data plane VCN **318**. The control plane VCN **316** and the data plane VCN **318** can be contained in a service tenancy **319** (e.g., the service tenancy **119** of FIG. **1**).

[0054] The control plane VCN **316** can include a control plane DMZ tier **320** (e.g., the control plane DMZ tier **120** of FIG. **1**) that can include load balancer (LB) subnet(s) **322** (e.g., LB subnet(s) **122** of FIG. **1**), a control plane app tier **324** (e.g., the control plane app tier **124** of FIG. **1**) that can include app subnet(s) **326** (e.g., similar to app subnet(s) **126** of FIG. **1**), and a control plane data tier **328** (e.g., the control plane data tier **128** of FIG. **1**) that can include DB subnet(s) **330**. The LB subnet(s) **322** contained in the control plane DMZ tier **320** can be communicatively coupled to the app subnet(s) **326** contained in the control plane app tier **324** and to an Internet gateway **334** (e.g., the Internet gateway **134** of FIG. **1**) that can be contained in the control plane VCN **316**. Additionally, the app subnet(s) **326** can be communicatively coupled to the DB subnet(s) **330** contained in the control plane data tier **328**, to a service gateway **336** (e.g., the service gateway of FIG. **1**), and a network address translation (NAT) gateway **338** (e.g., the NAT gateway **138** of FIG. **1**). The control plane VCN **316** can include the service gateway **336** and the NAT gateway **338**.

[0055] The data plane VCN **318** can include a data plane app tier **346** (e.g., the data plane app tier **146** of FIG. **1**), a data plane DMZ tier **348** (e.g., the data plane DMZ tier **148** of FIG. **1**), and a data plane data tier **350** (e.g., the data plane data tier **150** of FIG. **1**). The data plane DMZ tier **348** can include LB subnet(s) **322** that can be communicatively coupled to trusted app subnet(s) **360** and untrusted app subnet(s) **362** of the data plane app tier **346** and the Internet gateway **334** contained

in the data plane VCN **318**. The trusted app subnet(s) **360** can be communicatively coupled to the service gateway **336** contained in the data plane VCN **318**, the NAT gateway **338** contained in the data plane VCN **318**, and DB subnet(s) **330** contained in the data plane data tier **350**. The untrusted app subnet(s) **362** can be communicatively coupled to the service gateway **336** contained in the data plane VCN **318** and DB subnet(s) **330** contained in the data plane data tier **350**. The data plane data tier **350** can include DB subnet(s) **330** that can be communicatively coupled to the service gateway **336** contained in the data plane VCN **318**.

[0056] The untrusted app subnet(s) **362** can include one or more primary VNICs **364(1)-(N)** that can be communicatively coupled to tenant virtual machines (VMs) **366(1)-(N)**. Each tenant VM **366(1)-(N)** can be communicatively coupled to a respective app subnet **367(1)-(N)** that can be contained in respective container egress VCNs **368(1)-(N)** that can be contained in respective customer tenancies **380(1)-(N)**. Respective secondary VNICs **372(1)-(N)** can facilitate communication between the untrusted app subnet(s) **362** contained in the data plane VCN **318** and the app subnet contained in the container egress VCNs **368(1)-(N)**. Each container egress VCNs **368(1)-(N)** can include a NAT gateway **338** that can be communicatively coupled to public Internet **354** (e.g., public Internet **154** of FIG. 1).

[0057] The Internet gateway **334** contained in the control plane VCN **316** and contained in the data plane VCN **318** can be communicatively coupled to a metadata management service **352** (e.g., the metadata management service **152** of FIG. 1) that can be communicatively coupled to public Internet **354**. Public Internet **354** can be communicatively coupled to the NAT gateway **338** contained in the control plane VCN **316** and contained in the data plane VCN **318**. The service gateway **336** contained in the control plane VCN **316** and contained in the data plane VCN **318** can be communicatively couple to cloud services **356**.

[0058] In some embodiments, the data plane VCN **318** can be integrated with customer tenancies **380**. This integration can be useful or desirable for customers of the IaaS provider in some cases such as a case that may desire support when executing code. The customer may provide code to execute that may be destructive, may communicate with other customer resources, or may otherwise cause undesirable effects. In response to this, the IaaS provider may determine whether to execute code given to the IaaS provider by the customer.

[0059] In some examples, the customer of the IaaS provider may grant temporary network access to the IaaS provider and request a function to be attached to the data plane app tier **346**. Code to execute the function may be executed in the VMs **366(1)-(N)**, and the code may not be configured to execute anywhere else on the data plane VCN **318**. Each VM **366(1)-(N)** may be connected to one customer tenancy **380**. Respective containers **381(1)-(N)** contained in the VMs **366(1)-(N)** may be configured to execute the code. In this case, there can be a dual isolation (e.g., the containers **381(1)-(N)** executing code, where the containers **381(1)-(N)** may be contained in at least the VM **366(1)-(N)** that are contained in the untrusted app subnet(s) **362**), which may help prevent incorrect or otherwise undesirable code from damaging the network of the IaaS provider or from damaging a network of a different customer. The containers **381(1)-(N)** may be communicatively coupled to the customer tenancy **380** and may be configured to transmit or receive data from the customer tenancy **380**. The containers **381(1)-(N)** may not be configured to transmit or receive data from any other entity in the data plane VCN **318**. Upon completion of executing the code, the IaaS provider may kill or otherwise dispose of the containers **381(1)-(N)**.

[0060] In some embodiments, the trusted app subnet(s) **360** may execute code that may be owned or operated by the IaaS provider. In this embodiment, the trusted app subnet(s) **360** may be communicatively coupled to the DB subnet(s) **330** and be configured to execute CRUD operations in the DB subnet(s) **330**. The untrusted app subnet(s) **362** may be communicatively coupled to the DB subnet(s) **330**, but in this embodiment, the untrusted app subnet(s) may be configured to execute read operations in the DB subnet(s) **330**. The containers **381(1)-(N)** that can be contained in the VM **366(1)-(N)** of each customer and that may execute code from the customer may not be

communicatively coupled with the DB subnet(s) **330**.

[0061] In other embodiments, the control plane VCN **316** and the data plane VCN **318** may not be directly communicatively coupled. In this embodiment, there may be no direct communication between the control plane VCN **316** and the data plane VCN **318**. However, communication can occur indirectly through at least one method. An LPG **310** may be established by the IaaS provider that can facilitate communication between the control plane VCN **316** and the data plane VCN **318**. In another example, the control plane VCN **316** or the data plane VCN **318** can make a call to cloud services **356** via the service gateway **336**. For example, a call to cloud services **356** from the control plane VCN **316** can include a request for a service that can communicate with the data plane VCN **318**.

[0062] FIG. **4** is a block diagram illustrating another example pattern of an IaaS architecture **400** according to at least one embodiment. Service operators **402** (e.g., service operators **102** of FIG. **1**) can be communicatively coupled to a secure host tenancy **404** (e.g., the secure host tenancy **104** of FIG. **1**) that can include a virtual cloud network (VCN) **406** (e.g., the VCN **106** of FIG. **1**) and a secure host subnet **408** (e.g., the secure host subnet **108** of FIG. **1**). The VCN **406** can include an LPG **410** (e.g., the LPG **110** of FIG. **1**) that can be communicatively coupled to an SSH VCN **412** (e.g., the SSH VCN **112** of FIG. **1**) via an LPG **410** contained in the SSH VCN **412**. The SSH VCN **412** can include an SSH subnet **414** (e.g., the SSH subnet **114** of FIG. **1**). The SSH VCN **412** can be communicatively coupled to a control plane VCN **416** (e.g., the control plane VCN **116** of FIG. **1**) via an LPG **410** contained in the control plane VCN **416**. The SSH VCN **412** can be communicatively coupled to a data plane VCN **418** (e.g., the data plane VCN **118** of FIG. **1**) via an LPG **410** contained in the data plane VCN **418**. The control plane VCN **416** and the data plane VCN **418** can be contained in a service tenancy **419** (e.g., the service tenancy **119** of FIG. **1**).

[0063] The control plane VCN **416** can include a control plane DMZ tier **420** (e.g., the control plane DMZ tier **120** of FIG. **1**) that can include LB subnet(s) **422** (e.g., LB subnet(s) **122** of FIG. **1**), a control plane app tier **424** (e.g., the control plane app tier **124** of FIG. **1**) that can include app subnet(s) **426** (e.g., app subnet(s) **126** of FIG. **1**), and a control plane data tier **428** (e.g., the control plane data tier **128** of FIG. **1**) that can include DB subnet(s) **430** (e.g., DB subnet(s) **330** of FIG. **3**). The LB subnet(s) **422** contained in the control plane DMZ tier **420** can be communicatively coupled to the app subnet(s) **426** contained in the control plane app tier **424**. The LB subnet(s) **422** can be communicatively coupled to an Internet gateway **434** (e.g., the Internet gateway **134** of FIG. **1**) that can be contained in the control plane VCN **416**. The app subnet(s) **426** can be communicatively coupled to the DB subnet(s) **430** contained in the control plane data tier **428**, a service gateway **436** (e.g., the service gateway of FIG. **1**), and a network address translation (NAT) gateway **438** (e.g., the NAT gateway **138** of FIG. **1**). The control plane VCN **416** can include the service gateway **436** and the NAT gateway **438**.

[0064] The data plane VCN **418** can include a data plane app tier **446** (e.g., the data plane app tier **146** of FIG. **1**), a data plane DMZ tier **448** (e.g., the data plane DMZ tier **148** of FIG. **1**), and a data plane data tier **450** (e.g., the data plane data tier **150** of FIG. **1**). The data plane DMZ tier **448** can include LB subnet(s) **422** that can be communicatively coupled to trusted app subnet(s) **460** (e.g., trusted app subnet(s) **360** of FIG. **3**) and untrusted app subnet(s) **462** (e.g., untrusted app subnet(s) **362** of FIG. **3**) of the data plane app tier **446** and the Internet gateway **434** contained in the data plane VCN **418**. The trusted app subnet(s) **460** can be communicatively coupled to the service gateway **436** contained in the data plane VCN **418**, the NAT gateway **438** contained in the data plane VCN **418**, and DB subnet(s) **430** contained in the data plane data tier **450**. The untrusted app subnet(s) **462** can be communicatively coupled to the service gateway **436** contained in the data plane VCN **418** and DB subnet(s) **430** contained in the data plane data tier **450**. The data plane data tier **450** can include DB subnet(s) **430** that can be communicatively coupled to the service gateway **436** contained in the data plane VCN **418**.

[0065] The untrusted app subnet(s) **462** can include primary VNICS **464(1)-(N)** that can be

communicatively coupled to tenant virtual machines (VMs) **466(1)-(N)** residing within the untrusted app subnet(s) **462**. Each tenant VM **466(1)-(N)** can execute code in a respective container **467(1)-(N)** and be communicatively coupled to an app subnet **426** that can be contained in a data plane app tier **446** that can be contained in a container egress VCN **468**. Respective secondary VNICs **472(1)-(N)** can facilitate communication between the untrusted app subnet(s) **462** contained in the data plane VCN **418** and the app subnet contained in the container egress VCN **468**. The container egress VCN can include a NAT gateway **438** that can be communicatively coupled to public Internet **454** (e.g., public Internet **154** of FIG. 1).

[0066] The Internet gateway **434** contained in the control plane VCN **416** and contained in the data plane VCN **418** can be communicatively coupled to a metadata management service **452** (e.g., the metadata management service **152** of FIG. 1) that can be communicatively coupled to public Internet **454**. Public Internet **454** can be communicatively coupled to the NAT gateway **438** contained in the control plane VCN **416** and contained in the data plane VCN **418**. The service gateway **436** contained in the control plane VCN **416** and contained in the data plane VCN **418** can be communicatively couple to cloud services **456**.

[0067] In some examples, the pattern illustrated by the architecture of block diagram **400** of FIG. 4 may be considered an exception to the pattern illustrated by the architecture of block diagram **300** of FIG. 3. The pattern illustrated by the architecture of block diagram **400** of FIG. 4 may be implemented for a customer of the IaaS provider if the IaaS provider cannot directly communicate with the customer (e.g., a disconnected region). The respective containers **467(1)-(N)** that are contained in the VMs **466(1)-(N)** for each customer can be accessed in real-time by the customer. The containers **467(1)-(N)** may be configured to make calls to respective secondary VNICs **472(1)-(N)** contained in app subnet(s) **426** of the data plane app tier **446** that can be contained in the container egress VCN **468**. The secondary VNICs **472(1)-(N)** can transmit the calls to the NAT gateway **438** that may transmit the calls to public Internet **454**. In this example, the containers **467(1)-(N)** that can be accessed in real-time by the customer can be isolated from the control plane VCN **416** and can be isolated from other entities contained in the data plane VCN **418**. The containers **467(1)-(N)** may also be isolated from resources from other customers.

[0068] In other examples, the customer can use the containers **467(1)-(N)** to call cloud services **456**. In this example, the customer may execute code in the containers **467(1)-(N)** that requests a service from cloud services **456**. The containers **467(1)-(N)** can transmit this request to the secondary VNICs **472(1)-(N)** that can transmit the request to the NAT gateway that can transmit the request to public Internet **454**. Public Internet **454** can transmit the request to LB subnet(s) **422** contained in the control plane VCN **416** via the Internet gateway **434**. In response to determining the request is valid, the LB subnet(s) can transmit the request to app subnet(s) **426** that can transmit the request to cloud services **456** via the service gateway **436**.

[0069] It should be appreciated that IaaS architectures **100**, **200**, **300**, **400** depicted in the figures may have other components than those depicted. Further, the embodiments shown in the figures are only some examples of a cloud infrastructure system that may incorporate an embodiment of the disclosure. In some other embodiments, the IaaS systems may have more or fewer components than shown in the figures, may combine two or more components, or may have a different configuration or arrangement of components.

[0070] In certain embodiments, the IaaS systems described herein may include a suite of applications, middleware, and database service offerings that are delivered to a customer in a self-service, subscription-based, elastically scalable, reliable, highly available, and secure manner. An example of such an IaaS system is the Oracle Cloud Infrastructure (OCI) provided by the present assignee.

[0071] In one or more embodiments, a computer network provides connectivity among a set of nodes. The nodes may be local to and/or remote from each other. The nodes are connected by a set of links. Examples of links include a coaxial cable, an unshielded twisted cable, a copper cable, an

optical fiber, and a virtual link.

[0072] A subset of nodes implements the computer network. Examples of such nodes include a switch, a router, a firewall, and a network address translator (NAT). Another subset of nodes uses the computer network. Such nodes (also referred to as “hosts”) may execute a client process and/or a server process. A client process makes a request for a computing service (such as, execution of a particular application, and/or storage of a particular amount of data). A server process responds by executing the requested service and/or returning corresponding data.

[0073] A computer network may be a physical network, including physical nodes connected by physical links. A physical node is any digital device. A physical node may be a function-specific hardware device, such as a hardware switch, a hardware router, a hardware firewall, and a hardware NAT. Additionally, or alternatively, a physical node may be a generic machine that is configured to execute various virtual machines and/or applications performing respective functions. A physical link is a physical medium connecting two or more physical nodes. Examples of links include a coaxial cable, an unshielded twisted cable, a copper cable, and an optical fiber.

[0074] A computer network may be an overlay network. An overlay network is a logical network implemented on top of another network (such as a physical network). Each node in an overlay network corresponds to a respective node in the underlying network. Hence, each node in an overlay network is associated with both an overlay address (to address to the overlay node) and an underlay address (to address the underlay node that implements the overlay node). An overlay node may be a digital device and/or a software process (such as a virtual machine, an application instance, or a thread) A link that connects overlay nodes is implemented as a tunnel through the underlying network. The overlay nodes at either end of the tunnel treat the underlying multi-hop path between them as a single logical link. Tunneling is performed through encapsulation and decapsulation.

[0075] In an embodiment, a client may be local to and/or remote from a computer network. The client may access the computer network over other computer networks, such as a private network or the Internet. The client may communicate requests to the computer network using a communications protocol such as Hypertext Transfer Protocol (HTTP). The requests are communicated through an interface, such as a client interface (such as a web browser), a program interface, or an application programming interface (API).

[0076] In an embodiment, a computer network provides connectivity between clients and network resources. Network resources include hardware and/or software configured to execute server processes. Examples of network resources include a processor, a data storage, a virtual machine, a container, and/or a software application. Network resources are shared amongst multiple clients. Clients request computing services from a computer network independently of each other. Network resources are dynamically assigned to the requests and/or clients on an on-demand basis. Network resources assigned to each request and/or client may be scaled up or down based on, for example, (a) the computing services requested by a particular client, (b) the aggregated computing services requested by a particular tenant, and/or (c) the aggregated computing services requested of the computer network. Such a computer network may be referred to as a “cloud network.”

[0077] In an embodiment, a service provider provides a cloud network to one or more end users. Various service models may be implemented by the cloud network, including but not limited to Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS), and Infrastructure-as-a-Service (IaaS). In SaaS, a service provider provides end users the capability to use the service provider's applications that are executing on the network resources. In PaaS, the service provider provides end users the capability to deploy custom applications onto the network resources. The custom applications may be created using programming languages, libraries, services, and tools supported by the service provider. In IaaS, the service provider provides end users the capability to provision processing, storage, networks, and other fundamental computing resources provided by the network resources. Any arbitrary applications, including an operating system, may be deployed on the

network resources.

[0078] In an embodiment, various deployment models may be implemented by a computer network, including but not limited to a private cloud, a public cloud, and a hybrid cloud. In a private cloud, network resources are provisioned for exclusive use by a particular group of one or more entities (the term “entity” as used herein refers to a corporation, organization, person, or other entity). The network resources may be local to and/or remote from the premises of the particular group of entities. In a public cloud, cloud resources are provisioned for multiple entities that are independent from each other (also referred to as “tenants” or “customers”). The computer network and the network resources thereof are accessed by clients corresponding to different tenants. Such a computer network may be referred to as a “multi-tenant computer network.” Several tenants may use the same network resource at different times and/or at the same time. The network resources may be local to and/or remote from the premises of the tenants. In a hybrid cloud, a computer network comprises a private cloud and a public cloud. An interface between the private cloud and the public cloud allows for data and application portability. Data stored at the private cloud and data stored at the public cloud may be exchanged through the interface. Applications implemented at the private cloud and applications implemented at the public cloud may have dependencies on each other. A call from an application at the private cloud to an application at the public cloud (and vice versa) may be executed through the interface.

[0079] In an embodiment, tenants of a multi-tenant computer network are independent of each other. For example, a business or operation of one tenant may be separate from a business or operation of another tenant. Different tenants may demand different network requirements for the computer network. Examples of network requirements include processing speed, amount of data storage, security requirements, performance requirements, throughput requirements, latency requirements, resiliency requirements, Quality of Service (QOS) requirements, tenant isolation, and/or consistency. The same computer network may need to implement different network requirements demanded by different tenants.

[0080] In one or more embodiments in a multi-tenant computer network, tenant isolation is implemented to ensure that the applications and/or data of different tenants are not shared with each other. Various tenant isolation approaches may be used.

[0081] In an embodiment, each tenant is associated with a tenant ID. Each network resource of the multi-tenant computer network is tagged with a tenant ID. A tenant is permitted access to a particular network resource only if the tenant and the particular network resources are associated with the same tenant ID.

[0082] In an embodiment, each tenant is associated with a tenant ID. Each application, implemented by the computer network, is tagged with a tenant ID. Additionally, or alternatively, each data structure and/or dataset stored by the computer network is tagged with a tenant ID. A tenant is permitted access to a particular application, data structure, and/or dataset only if the tenant and the particular application, data structure, and/or dataset are associated with the same tenant ID.

[0083] As an example, each database implemented by a multi-tenant computer network may be tagged with a tenant ID. Only a tenant associated with the corresponding tenant ID may access data of a particular database. As another example, each entry in a database implemented by a multi-tenant computer network may be tagged with a tenant ID. Only a tenant associated with the corresponding tenant ID may access data of a particular entry. However, the database may be shared by multiple tenants.

[0084] In an embodiment, a subscription list indicates the tenants that have authorization to access an application. For each application, a list of tenant IDs of tenants authorized to access the application is stored. A tenant is permitted access to a particular application only if the tenant ID of the tenant is included in the subscription list corresponding to the particular application.

[0085] In an embodiment, network resources (such as digital devices, virtual machines, application instances, and threads) corresponding to different tenants are isolated to tenant-specific overlay

networks maintained by the multi-tenant computer network. As an example, packets from any source device in a tenant overlay network may only be transmitted to other devices within the same tenant overlay network. Encapsulation tunnels are used to prohibit any transmissions from a source device on a tenant overlay network to devices in other tenant overlay networks. Specifically, the packets received from the source device are encapsulated within an outer packet. The outer packet is transmitted from a first encapsulation tunnel endpoint (in communication with the source device in the tenant overlay network) to a second encapsulation tunnel endpoint (in communication with the destination device in the tenant overlay network). The second encapsulation tunnel endpoint decapsulates the outer packet to obtain the original packet transmitted by the source device. The original packet is transmitted from the second encapsulation tunnel endpoint to the destination device in the same particular overlay network.

4. COMPUTER SYSTEM

[0086] FIG. 5 illustrates an example computer system **500**, where various embodiments may be implemented. The system **500** may be used to implement any of the computer systems described above. As shown in FIG. 5, computer system **500** includes a processing unit **504** that communicates with several peripheral subsystems via a bus subsystem **502**. These peripheral subsystems may include a processing acceleration unit **506**, an I/O subsystem **508**, a storage subsystem **518**, and a communications subsystem **524**. Storage subsystem **518** includes tangible computer-readable storage media **522** and a system memory **510**.

[0087] Bus subsystem **502** provides a mechanism for letting the various components and subsystems of computer system **500** communicate with each other as intended. Although bus subsystem **502** is shown schematically as a single bus, alternative embodiments of the bus subsystem may utilize multiple buses. Bus subsystem **502** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, and a local bus using any of a variety of bus architectures. For example, such architectures may include an Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnect (PCI) bus. The PCI bus can be implemented as a Mezzanine bus manufactured to the IEEE P1386.1 standard.

[0088] Processing unit **504** that can be implemented as one or more integrated circuits (e.g., a conventional microprocessor or microcontroller) controls the operation of computer system **500**. One or more processors may be included in processing unit **504**. These processors may include single core or multicore processors. In certain embodiments, processing unit **504** may be implemented as one or more independent processing units **532** and/or **534** with single or multicore processors included in each processing unit. In other embodiments, processing unit **504** may also be implemented as a quad-core processing unit formed by integrating two dual-core processors into a single chip.

[0089] In various embodiments, processing unit **504** can execute a variety of programs in response to program code and can maintain multiple concurrently executing programs or processes. At any given time, some of the program code to be executed can be resident in processing unit **504** and/or in storage subsystem **518**. Through suitable programming, processing unit **504** can provide various functionalities described above. Computer system **500** may additionally include a processing acceleration unit **506** that can include a digital signal processor (DSP), a special-purpose processor, and/or the like.

[0090] I/O subsystem **508** may include user interface input devices and user interface output devices. User interface input devices may include a keyboard, pointing devices such as a mouse or trackball, a touchpad or touch screen incorporated into a display, a scroll wheel, a click wheel, a dial, a button, a switch, a keypad, audio input devices with voice command recognition systems, microphones, and other types of input devices. User interface input devices may include, for example, motion sensing and/or gesture recognition devices such as the Microsoft Kinect® motion

sensor that enables users to control and interact with an input device, such as the Microsoft Xbox® 360 game controller, through a natural user interface using gestures and spoken commands. User interface input devices may also include eye gesture recognition devices such as the Google Glass® blink detector that detects eye activity (e.g., ‘blinking’ while taking pictures and/or making a menu selection) from users and transforms the eye gestures as input into an input device (e.g., Google Glass®). Additionally, user interface input devices may include voice recognition sensing devices that enable users to interact with voice recognition systems (e.g., Siri® navigator) through voice commands.

[0091] User interface input devices may also include, without limitation, three dimensional (3D) mice, joysticks or pointing sticks, gamepads and graphic tablets, and audio/visual devices such as speakers, digital cameras, digital camcorders, portable media players, webcams, image scanners, fingerprint scanners, barcode reader 3D scanners, 3D printers, laser rangefinders, and eye gaze tracking devices. Additionally, user interface input devices may include, for example, medical imaging input devices such as computed tomography, magnetic resonance imaging, position emission tomography, and medical ultrasonography devices. User interface input devices may also include, for example, audio input devices such as MIDI keyboards, digital musical instruments, and the like.

[0092] User interface output devices may include a display subsystem, indicator lights, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device, such as that using a liquid crystal display (LCD) or plasma display, a projection device, a touch screen, and the like. In general, use of the term “output device” is intended to include all possible types of devices and mechanisms for outputting information from computer system **500** to a user or other computer. For example, user interface output devices may include, without limitation, a variety of display devices that visually convey text, graphics, and audio/video information such as monitors, printers, speakers, headphones, automotive navigation systems, plotters, voice output devices, and modems.

[0093] Computer system **500** may comprise a storage subsystem **518** that provides a tangible, non-transitory, computer-readable storage medium for storing software and data constructs that provide the functionality of the embodiments described in this disclosure. The software can include programs, code modules, instructions, scripts, etc., that when executed by one or more cores or processors of processing unit **504**, provide the functionality described above. Storage subsystem **518** may also provide a repository for storing data used in accordance with the present disclosure.

[0094] As depicted in the example in FIG. 5, storage subsystem **518** can include various components including a system memory **510**, computer-readable storage media **522**, and a computer readable storage media reader **520**. System memory **510** may store program instructions, such as application programs **512**, that are loadable and executable by processing unit **504**. System memory **510** may also store data, such as program data **514**, that is used during the execution of the instructions and/or data that is generated during the execution of the program instructions. Various different kinds of programs may be loaded into system memory **510** including but not limited to client applications, Web browsers, mid-tier applications, relational database management systems (RDBMS), virtual machines, containers, etc.

[0095] System memory **510** may also store an operating system **516**. Examples of operating system **516** may include various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems, a variety of commercially-available UNIX® or UNIX-like operating systems (including without limitation the variety of GNU/Linux operating systems, the Google Chrome® OS, and the like) and/or mobile operating systems such as iOS, Windows® Phone, Android® OS, BlackBerry® OS, and Palm® OS operating systems. In certain implementations, where computer system **500** executes one or more virtual machines, the virtual machines along with their guest operating systems (GOSs) may be loaded into system memory **510** and executed by one or more processors or cores of processing unit **504**.

[0096] System memory **510** can come in different configurations depending upon the type of computer system **500**. For example, system memory **510** may be volatile memory (such as random access memory (RAM)) and/or non-volatile memory (such as read-only memory (ROM), flash memory, etc.). Different types of RAM configurations may be provided including a static random access memory (SRAM), a dynamic random access memory (DRAM), and others. In some implementations, system memory **510** may include a basic input/output system (BIOS) containing basic routines that help to transfer information between elements within computer system **500** such as during start-up.

[0097] Computer-readable storage media **522** may represent remote, local, fixed, and/or removable storage devices plus storage media for temporarily and/or more permanently containing, storing, computer-readable information for use by computer system **500** including instructions executable by processing unit **504** of computer system **500**.

[0098] Computer-readable storage media **522** can include any appropriate media known or used in the art, including storage media and communication media, such as but not limited to, volatile and non-volatile, removable and non-removable media implemented in any method or technology for storage and/or transmission of information. This can include tangible computer-readable storage media such as RAM, ROM, electronically erasable programmable ROM (EEPROM), flash memory or other memory technology, CD-ROM, digital versatile disk (DVD), or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or other tangible computer readable media.

[0099] By way of example, computer-readable storage media **522** may include a hard disk drive that reads from or writes to non-removable, nonvolatile magnetic media, a magnetic disk drive that reads from or writes to a removable, nonvolatile magnetic disk, and an optical disk drive that reads from or writes to a removable, nonvolatile optical disk such as a CD ROM, DVD, and Blu-Ray® disk, or other optical media. Computer-readable storage media **522** may include, but is not limited to, Zip® drives, flash memory cards, universal serial bus (USB) flash drives, secure digital (SD) cards, DVD disks, digital video tape, and the like. Computer-readable storage media **522** may also include solid-state drives (SSD) based on non-volatile memory such as flash-memory based SSDs, enterprise flash drives, solid state ROM, and the like, SSDs based on volatile memory such as solid state RAM, dynamic RAM, static RAM, DRAM-based SSDs, magnetoresistive RAM (MRAM) SSDs, and hybrid SSDs that use a combination of DRAM and flash memory based SSDs. The disk drives and their associated computer-readable media may provide non-volatile storage of computer-readable instructions, data structures, program modules, and other data for computer system **500**.

[0100] Machine-readable instructions executable by one or more processors or cores of processing unit **504** may be stored on a non-transitory computer-readable storage medium. A non-transitory computer-readable storage medium can include physically tangible memory or storage devices that include volatile memory storage devices and/or non-volatile storage devices. Examples of non-transitory computer-readable storage medium include magnetic storage media (e.g., disk or tapes), optical storage media (e.g., DVDs, CDs), various types of RAM, ROM, or flash memory, hard drives, floppy drives, detachable memory drives (e.g., USB drives), or other type of storage device.

[0101] Communications subsystem **524** provides an interface to other computer systems and networks. Communications subsystem **524** serves as an interface for receiving data from and transmitting data to other systems from computer system **500**. For example, communications subsystem **524** may enable computer system **500** to connect to one or more devices via the Internet. In some embodiments, communications subsystem **524** can include radio frequency (RF) transceiver components for accessing wireless voice and/or data networks (e.g., using cellular telephone technology, advanced data network technology, such as 3G, 4G or EDGE (enhanced data rates for global evolution), WiFi (IEEE 802.11 family standards, or other mobile communication technologies, or any combination thereof), global positioning system (GPS) receiver components, and/or other components. In some embodiments, communications subsystem **524** can provide

wired network connectivity (e.g., Ethernet) in addition to or instead of a wireless interface.

[0102] In some embodiments, communications subsystem **524** may also receive input communication in the form of structured and/or unstructured data feeds **526**, event streams **528**, event updates **530**, and the like on behalf of one or more users who may use computer system **500**.

[0103] By way of example, communications subsystem **524** may be configured to receive data feeds **526** in real-time from users of social networks and/or other communication services such as Twitter® feeds, Facebook® updates, web feeds such as Rich Site Summary (RSS) feeds, and/or real-time updates from one or more third party information sources.

[0104] Additionally, communications subsystem **524** may also be configured to receive data in the form of continuous data streams that may include event streams **528** of real-time events and/or event updates **530** that may be continuous or unbounded in nature with no explicit end. Examples of applications that generate continuous data may include, for example, sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), clickstream analysis tools, automobile traffic monitoring, and the like.

[0105] Communications subsystem **524** may also be configured to output the structured and/or unstructured data feeds **526**, event streams **528**, event updates **530**, and the like to one or more databases that may be in communication with one or more streaming data source computers coupled to computer system **500**.

[0106] Computer system **500** can be one of various types, including a handheld portable device (e.g., an iPhone® cellular phone, an iPad® computing tablet, a PDA), a wearable device (e.g., a Google Glass® head mounted display), a PC, a workstation, a mainframe, a kiosk, a server rack, or any other data processing system.

[0107] Due to the ever-changing nature of computers and networks, the description of computer system **500** depicted in FIG. **5** is intended only as a specific example. Many other configurations having more or fewer components than the system depicted in FIG. **5** are possible. For example, customized hardware might also be used and/or particular elements might be implemented in hardware, firmware, software (including applets), or a combination. Further, connection to other computing devices, such as network input/output devices, may be employed. Based on the disclosure and teachings provided herein, a person of ordinary skill in the art will appreciate other ways and/or methods to implement the various embodiments.

4. CONTAINER ORCHESTRATION SYSTEM

[0108] A detailed example is described below for purposes of clarity. Components and/or operations described below should be understood as one specific example may not be applicable to certain embodiments. Accordingly, components and/or operations described below should not be construed as limiting the scope of any of the claims.

[0109] FIG. **6** illustrates a system **600** in accordance with one or more embodiments. As illustrated in FIG. **6**, system **600** includes service tenancies **602A** and **602B**, customer tenancy **604**, host **603A** and **603B**, virtual nodes including a virtual agent (virtual agent replicas **606A** and **606B**) and container pods **620A-620D**, container orchestration control plane **608**, container orchestration API server **610**, service VNIC **612**, primary VNIC **614A** and **614B**, container instance service **630**, container instance control plane **616**, container instances **618A** and **618B**, and kube-proxy **622A** and **622B**.

[0110] In one or more embodiments, system **600** may include more or fewer components than the components illustrated in FIG. **6**. The components illustrated in FIG. **6** may be local to or remote to the others. The components illustrated in FIG. **6** may be implemented in software and/or hardware. Components may be distributed over multiple applications and/or machines. Multiple components may be combined into one application and/or machine. Operations described with respect to one component may instead be performed by another component.

[0111] In accordance with an embodiment, a tenancy is a secure and isolated partition within a cloud system that allows a tenant to create, organize, and administer their cloud resources. In one

or more embodiments, a tenancy is a hierarchical collection of compartments, where the root compartment is the tenancy. A tenant, or customer, is a party with a tenancy in the cloud system. A cloud system includes multiple tenancies that are isolated from each other. A cloud network manager is manager for one or more tenants or customers in a cloud network, such as an owner or renter of the cloud network.

[0112] In accordance with an embodiment, service tenancies **602A** and **602B** are tenancies under the control of the cloud network manager. Components in service tenancies, such as virtual agents (virtual agent replicas **606A** and **606B**), are version patched under the control of the cloud network without requiring a request from the customer. Furthermore, the components in the service tenancies are protected using cloud network security.

[0113] In accordance with an embodiment, customer tenancy **604** is a tenancy under the control of the customer. Customer tenancy **604** contains a customer network such as a customer network for a container orchestration cluster. In the example of FIG. 6, service tenancies **602A** and **602B** and customer tenancy **604** are implemented within the same cloud environment and configured to execute operations corresponding to a data set associated with the customer. In an example, the data set defines a container orchestration cluster such as a Kubernetes cluster.

[0114] In accordance with an embodiment, hosts **603A** and **603B** are separate computers or devices that connect to the network. Hosts **603A** and **603B** are in different fault domains. A fault domain is a group of nodes that share physical infrastructure. In an example, a particular node is associated with one more fault domains; including include regions (e.g., a geographical area, such as a city), availability zones (partitioning within a region with dedicated power and cooling), or other fine-grained partitioning of a physical infrastructure (e.g., a semi-isolated rack within a data center).

[0115] In accordance with an embodiment, a container orchestration system provides a runtime for containerized workloads and services. Examples of container orchestration systems include Kubernetes and Docker Swarm. A container orchestration environment is an instance of a container orchestration system. A specific Kubernetes cluster or a specific Docker Swarm instance are examples of container orchestration environments.

[0116] In accordance with an embodiment, a container orchestration implementation provider is an implementation provider for a particular type of container orchestration system. Examples of container orchestration implementations include Oracle Container Engine for Kubernetes (OKE) and Amazon Elastic Kubernetes Service (EKS) that both provide container orchestration implementations (i.e., are vendors) for Kubernetes.

[0117] In accordance with an embodiment, a container orchestration node is a virtual or physical machine in a container orchestration cluster. A control plane manages the container orchestration nodes and contains the services necessary to execute containers or pods. For a Kubernetes cluster, the container orchestration node is a Kubernetes node. Components on a container orchestration node in Kubernetes include a kubelet, a container runtime, and a kube-proxy. A container orchestration node is an individual bare metal machine or virtual machine (VM), where containers are scheduled to execute within a container orchestration environment, for example, as part of a Kubernetes cluster or Docker Swarm instance.

[0118] In accordance with an embodiment, a container orchestration agent executes on container orchestration nodes and is responsible for communications between the container orchestration control plane and the node where the workload is executed. In Kubernetes, the container orchestration agent is a kubelet.

[0119] In accordance with an embodiment, a virtual node is a container orchestration node implemented on multiple hosts, computers, or devices. A virtual agent is a container orchestration agent for a virtual node. The virtual agent is executed on a host, computer, or device different than containers for the virtual node. The virtual agent interacts with containers, such as containers in pods **620A-D** at container instances **618A** and **618B**. The virtual agent and the containers execute at separate locations within the virtual node.

[0120] In accordance with an embodiment, virtual agent replicas **606A** and **606B** are replicas of a virtual agent for a virtual node. Multiple virtual agent replicas **606A** and **606B** allow the virtual node to be operated in a high availability manner. Virtual agent replicas **606A** and **606B** are executed on different hosts, such as hosts **603A** and **603B**, on different fault domains to provide for high availability for the virtual agent and virtual node. For example, if virtual agent replica **606A** fails, virtual agent replica **606B** maintains operation as a virtual agent in the virtual node. Virtual agents provide customers with the ability to deploy containerized applications without having to manage the data plane infrastructure. Thus, the virtual agent reduces the operational burden on the customer.

[0121] In accordance with an embodiment, a container instance, such as container instances **618A** and **618B**, is a virtual machine that executes a containerized application in a cloud system. A container instance provides the benefits of a traditional VM instance, such as CPU and memory resources. The container instance uses a standardized and/or reduced functionality for containers. In the example of FIG. 6, container instances **618A** and **618B** include container pods **620A-D** and kube-proxies **622A** and **622B**.

[0122] In accordance with an embodiment, container pods, such as container pods **620A-D**, execute containers scheduled on a virtual node. A Kubernetes pod is a group of one or more containers with shared storage and network resources and a specification for how to execute the containers. In the example of FIG. 6, container pods **620A-D** are part of a single virtual node along with virtual agent replicas **606A** and **606B**.

[0123] In accordance with an embodiment, a network proxy performs stream forwarding and includes other functionality such as filtering content, scanning for malware, masking the origin of the requests, and encryption. A Kubernetes network kube-proxy executes on the Kubernetes nodes and includes functionality for simple TCP, UDP, and SCTP stream forwarding or round robin TCP, UDP, and SCTP forwarding. Kube Proxy routes traffic headed to container orchestration service endpoints. In the example of FIG. 6, a virtual node uses multiple container instances in different locations, so multiple kube-proxies **622A** and **622B** are used in a single virtual node, and container instances **618A** and **618B** both contain a kube proxy as a sidecar. Containers in a container instance share the same network namespace and, in that case, two containers in a container instance cannot use the same port.

[0124] In accordance with an embodiment, container orchestration control plane **608** acts as the control plane for a container orchestration cluster. In Kubernetes, a Kubernetes control plane makes global decisions about the cluster, including scheduling, detects cluster events, and responds to cluster events (for example, starting up new pods).

[0125] In accordance with an embodiment, container orchestration API server **610** exposes an API to allow users to control the cluster. In Kubernetes, the Kubernetes API server is a component of the Kubernetes control plane that exposes the Kubernetes API and is the front end for the Kubernetes control plane. The Kubernetes API is a resource-based (RESTful) programmatic interface provided via HTTP. The Kubernetes API supports retrieving, creating, updating, and deleting primary resources via the standard HTTP verbs (POST, PUT, PATCH, DELETE, GET).

[0126] In accordance with an embodiment, container instance service **630** is a service that contains container instance control plane **616** and container instances **618A** and **618B**. Container instance service **630** creates and maintains container instances, such as container instances **618A** and **618B**.

[0127] In accordance with an embodiment, container instance control plane **616** is a layer that handles tasks for container instances. Container instance control plane **616** configures network devices, allocates IP addresses, manages network security, and creates and distributes routing policies.

[0128] In accordance with an embodiment, a Virtual Network Interface Card (VNIC) enables elements to connect to a cloud network in a cloud environment. A VNIC is an abstraction for one or more physical Network Interface Cards (NICs) in the cloud network. A NIC, also known as a

network adapter, LAN adapter, or physical network interface, is a computer hardware component that connects computers to a computer network.

[0129] In the example of FIG. 6, service VNIC **612** connects to virtual agent replicas **606A** and **606B** using a single address and load balances requests between virtual agent replicas **606A** and **606B**. Service VNIC **612** attaches the virtual agent to the customer network with a single address for virtual agent replicas **606A** and **606B** executing on different hosts **603A** and **603B**. A host executes a set of virtual agents belonging to different tenants.

[0130] In accordance with an embodiment, primary VNICs **614A** and **614B** connect virtual agent replicas **606A** and **606B** to other elements in the cloud environment, such as container instance control plane **616**. Primary VNIC **614A** is associated with virtual agent replica **606A**, and primary VNIC **614B** is associated with virtual agent replica **606B**.

[0131] In accordance with an embodiment, control plane APIs create a virtual node pool, defined as a collection of virtual nodes including virtual agents. Customers interact with the container orchestration cluster using container orchestration API server **610**. Customers create pods, such as container pods **620A-D**, for a virtual node by storing an update using container orchestration API server **610**. The virtual agent of the virtual node obtains information from container orchestration API server **610** and provision containers for the pod, such as container pods **620A-D**, at container instances, such as container instances **618A** and **618B**. Customers may retrieve logs from a pod using the virtual agent.

[0132] In accordance with an embodiment, the virtual agent generates a container instance, such as one of container instances **618A** or **618B**, for a pod scheduled on the virtual node. A service tenancy executes container instances **618A** or **618B**. The container instances **618A** and **618B** are invisible to customers, but the customer network connects to container pods **620A-D** in container instances **618A** or **618B**. For example, customers may access applications executing in a pod of a container instance using the pods IP address in the customer network.

[0133] In accordance with an embodiment, virtual agents use an IP address for communication with the container orchestration cluster. A management plane provisions service VNIC **612** in a customer's network to obtain this IP address. The management plane creates a primary VNIC, such as primary VNIC **614A** and service VNIC **612**, for the virtual agent replica to connect to container instance control plane **616**. The service VNIC **612** routes service VNIC Network traffic from virtual agent replicas **606A** and **606B** by setting the source IP to the Service VNIC IP. Service VNIC **612** routes inbound network traffic to virtual agent replicas **606A** and **606B** to one of the virtual agent replicas **606A** or **606B**.

[0134] In accordance with an embodiment, the virtual agent (such as virtual agent replicas **606A** and **606B**) includes a node controller responsible for registering the virtual agent with the container orchestration API server and periodically heart-beating its status. The virtual agent schedules pods on a healthy node as indicated by node status. Additionally, or alternatively, the virtual agent includes a pod controller responsible for managing the pods scheduled on the virtual node by provisioning container instances. Additionally, or alternatively, the virtual agent includes a virtual agent API server that implements APIs for extracting container logs and executing remote commands on containers.

[0135] In one embodiment, one of the virtual node replicas **606A** and **606B**, called a leader replica, executes the node controller and manages container instances. The other replica, called a follower replica, responds to container orchestration requests and acts as a backup. In one example, Kubernetes Leader Election library, a standard Kubernetes feature that uses lease objects, is used for leader election. In one or more embodiments, the lease object is marked as a child of a node object, so it is cleaned up when the node containing the leader replica is deleted.

[0136] In one or more embodiments, the virtual agents use credentials provided by a management plane. The provided certificates include a client certificate to communicate with container orchestration API server for registering nodes, updating node status, retrieving pod information,

and updating pod status as well as a server certificate. The virtual agent is provided with a server certificate, signed by cluster certificate authority, by the management plane. Container orchestration API server verifies this certificate to ensure that the server certificate is signed by cluster certificate authority to establish trust. The management plane also provides the virtual agent with a client certificate signed by the cluster certificate authority.

[0137] In an embodiment, the management plane ensures that virtual agents execute the right version using a transparent upgrade. The management plane implements a rolling upgrade of virtual agent replicas **606A** and **606B** without any downtime to customers. The upgrades include upgrades necessitated by container orchestration version upgrade. Customers may trigger a container orchestration version upgrade on a cluster using a control plane API to automatically upgrade virtual agents in the cluster if required. The upgrades also include routine bug fixes and enhancements deployed automatically in the background.

[0138] In one or more embodiments, a data repository stores the data and configuration of system **600**. The data repository may be any type of storage unit and/or device (e.g., a file system, database, collection of tables, or any other storage mechanism) for storing data. Further, a data repository may include multiple different storage units and/or devices. The multiple different storage units and/or devices may or may not be of the same type or located at the same physical site. Further, a data repository may be implemented or executed on the same computing system as system **600**. Additionally, or alternatively, a data repository may be implemented or executed on a computing system separate from system **600**. The data repository may be communicatively coupled via a direct connection or via a network. Information describing system **600** may be implemented across any of the components within the system **600**.

[0139] In one or more embodiments, service tenancies **602A** and **602B**, customer tenancy **604**, host **603A** and **603B**, virtual nodes including virtual agent (virtual agent replica **606A** and **606B**) and container pods **620A-620D**, container orchestration control plane **608**, container orchestration API server **610**, service VNIC **612**, primary VNICs **614A** and **614B**, container instance control plane **616**, container instances **618A** and **618B**, and kube-proxy **622A** and **622B** refer to hardware and/or software configured to perform operations described herein for container orchestration. Examples of operations for container orchestration are described below with reference to FIGS. **7** and **8**.

[0140] In an embodiment, service tenancies **602A** and **602B**, customer tenancy **604**, host **603A** and **603B**, virtual nodes including virtual agent (virtual agent replica **606A** and **606B**) and container pods **620A-620D**, container orchestration control plane **608**, container orchestration API server **610**, service VNIC **612**, primary VNICs **614A** and **614B**, container instance control plane **616**, container instances **618A** and **618B**, and kube-proxy **622A** and **622B** are implemented on one or more digital devices. The term “digital device” generally refers to any hardware device that includes a processor. A digital device may refer to a physical device executing an application or a virtual machine. Examples of digital devices include a computer, a tablet, a laptop, a desktop, a netbook, a server, a web server, a network policy server, a proxy server, a generic machine, a function-specific hardware device, a hardware router, a hardware switch, a hardware firewall, a hardware firewall, a hardware network address translator (NAT), a hardware load balancer, a mainframe, a television, a content receiver, a set-top box, a printer, a mobile handset, a smartphone, a personal digital assistant (PDA), a wireless receiver and/or transmitter, a base station, a communication management device, a router, a switch, a controller, an access point, and/or a client device.

5. CONTAINER ORCHESTRATION METHOD

[0141] FIG. **7** illustrates an example set of operations for preparing a virtual agent in a cloud environment in accordance with one or more embodiments. One or more operations illustrated in FIG. **7** may be modified, rearranged, or omitted. Accordingly, the particular sequence of operations illustrated in FIG. **7** should not be construed as limiting the scope of one or more embodiments.

[0142] In an embodiment, the system executes a virtual agent with multiple virtual agent replicas

for a virtual node on first service tenancy (Operation **702**). The system uses computers, bare metal servers, and/or virtual machines to execute the functionality of the virtual agent on the first service tenancy. Since the first service tenancy executes the virtual agent, the virtual agent is maintained and updated by a cloud network manager rather than the customer being required to maintain and update the virtual agent. The system patches a version of the virtual agent in the first service tenancy under the control of the cloud network without requiring a request from the customer. Furthermore, the virtual agent in the first service tenancy is protected using cloud network security for the first service tenancy.

[0143] In an embodiment, the system executes a container orchestration API server on second service tenancy (Operation **704**). The second service tenancy executes a container orchestration control plane including the container orchestration API server for a container orchestration cluster. The system uses computers, bare metal servers, and/or virtual machines to execute the functionality of the container orchestration API server on the second service tenancy. Since the second service tenancy executes the container orchestration API server, a cloud network manager maintains and updates the container orchestration API server rather than require the customer to maintain and update the virtual agent. The cloud network manager updates the container orchestration version, such as the Kubernetes version, of the container orchestration API server. Furthermore, the container orchestration API server in the second service tenancy is protected using cloud network security for the second service tenancy.

[0144] In an embodiment, the system executes a customer network including service VNIC interfacing with the virtual agent replicas (Operation **706**). The customer network in the customer tenancy is under the control of the customer. The service VNIC load balances between the virtual agent replicas.

[0145] In an embodiment, the system authorizes the virtual agent to the container instance control plane (Operation **708**). Authentication is the process of verifying a unit such as the virtual agent. Authentication involves validating credentials using local databases or external authentication servers (such as Automatic Configuration Servers). After successful authentication, the system uses authorization to determine what resources the virtual agent is allowed to access and the operations they are able to perform. The authorization ensures that the virtual agent is limited to operating on behalf of a single customer as defined by a subnet in the cloud network.

6. CONTAINER ORCHESTRATION SIGNALING

[0146] FIG. **8** illustrates an example set of operations for container orchestration using a virtual agent in a cloud environment in accordance with one or more embodiments. One or more operations illustrated in FIG. **8** may be modified, rearranged, or omitted. Accordingly, the particular sequence of operations illustrated in FIG. **8** should not be construed as limiting the scope of one or more embodiments.

[0147] In an embodiment, the system checks if container orchestration API server is updated (Operation **802**). For example, a user may indicate that a container is to be created, updated, or deleted from a container orchestration cluster using the container orchestration API server. The container orchestration API server then stores the update in a database.

[0148] In an embodiment, if the container orchestration API server is updated, the system sends a request to virtual agent through the customer tenancy (Operation **804**). The request concerns a container scheduled on a virtual node. The system sends the request to the virtual agent of the node such as to create, modify, or delete a container of the container orchestration cluster. The request passes through the customer tenancy as part of the container orchestration cluster. The system routes a request from the container orchestration API server to the virtual agent through the customer tenancy of a customer of the cloud network manager. By routing the request through the customer tenancy, the system uses the customer network, while the container orchestration API server and virtual agent remain in a service tenancy (and thus, are maintained and updated by the cloud network manager). The system routes the request through the customer tenancy by

addressing the request to a service VNIC in the customer tenancy that sends the request to the virtual agent.

[0149] In one embodiment, the request creates, updates, or destroys containers for the virtual node. The container orchestration API server sends the request through the customer tenancy and is rerouted to the virtual agent in the first service tenancy with the service VNIC.

[0150] In an embodiment, the service VNIC load balances the request and sends the request to one of the virtual agent replicas at different hosts if the container orchestration (Operation **806**). The service VNIC distributes incoming requests across multiple virtual agent replicas to make sure that the virtual agent replicas are able to handle the requests. In this way, the service VNIC distributes requests to improve speed and capacity utilization, preventing any single server from being overworked. By load balancing, the service VNIC provides for high availability and reliability because if one virtual agent replica fails, the service VNIC redirects requests to other virtual agent replica. The service VNIC uses an algorithm to load balance the requests, such as round robin or least utilization. The service VNIC uses a single address for the virtual agent with the existence of the multiple virtual agent replicas and the virtualization of the virtual node being hidden from the customer network and container orchestration API server.

[0151] In accordance with an embodiment, one of the virtual agent replicas receives the request and sends a message to the container instance control plane to update pods in a container instance (Operation **808**). The virtual agent replica, or an associated module, keeps track of the pods and container instances used in the virtual node, and the virtual agent replica sends the request identifying the pod and container instance. The container instance control plane then creates, updates, or deletes pods or container instances as needed. In one example, the virtual agent instructs a container instance control plane to create, update, or destroy container instances containing container orchestration pods.

[0152] In accordance with an embodiment, virtual agent replica initialization involves warming a container instance cache with the container instances launched by the virtual agent. The system filters on a display name of the container instance and makes a list call to the container instance control plane. In one example, the display name is set to the virtual agent ID.

[0153] In accordance with an embodiment, a virtual agent process executes an admin endpoint to serve health probes that are used to monitor the virtual node health. The admin endpoint is an Hypertext Transfer Protocol (HTTP) listener accessible via the bridge network within the data plane host.

[0154] In accordance with an embodiment, Kubernetes pod spec is mapped to a container instance such that the containers in the pod have a corresponding container in container instance. Kube-Proxy containers are added to the pod spec containers. Whenever a new pod is scheduled by the Kubernetes scheduler on a virtual node, the virtual-agent receives the pod-spec. The system transforms the pod-spec into a container instance API request to launch a container instance.

[0155] In accordance with an embodiment, to keep track of container instances managed by the virtual agent, the display name of container instance is set to virtual agent system identifier (ID). When a virtual agent process starts up, the virtual agent process queries for container instances using a display name filter to retrieve the container instances managed by the virtual agent. In a virtual agent process start up process of one embodiment, the management plane invokes the data plane host to execute a virtual agent replica. The data plane host then starts the virtual agent container and configures container network namespace. The virtual agent registers the node with the container orchestration API server. A service VNIC sends packets to the container orchestration API server. The packets flow in the customer's network in the customer tenancy. The virtual agent retrieves a list of container instances managed by this virtual agent from a container instance control plane and populates a host-local cache (if there was another virtual agent replica before this virtual agent process, there may be some container instances provisioned earlier). When a cluster user uses container orchestration API command, such as a “kubectl” command for Kubernetes, to

list nodes in the cluster, the system includes the virtual agent in the list of nodes. The IP address of the virtual agent is set to the Service VNIC IP address.

[0156] In accordance with an embodiment, when a cluster user deploys a pod to the virtual agent, the virtual agent discovers that a new pod needs to be provisioned. The virtual agent uses the container instance control plane to launch a container instance. The container instance is in service tenancy but attached to customer's network. The container instance control plane provisions the container instance and virtual agent sets the pod status to pending. Once a container instance is successfully provisioned, the virtual agent calls the container orchestration API Server to set the IP address of the pod to container instance IP address and the pod status is set to running. The cluster user determines the new pod is executing when using container orchestration API command, such as a “kubectl” command for Kubernetes, to list pods in the cluster.

[0157] In accordance with an embodiment, when the customer retrieves logs from the pod, a cluster user uses a container orchestration API command, such as a “kubectl” command for Kubernetes, to retrieve container logs from the pod. The container instance API server sends a request to virtual agent to retrieve logs. A virtual agent invokes container instance control plane API to retrieve container logs. The container instance control plane retrieves container logs from the container instance. The virtual agent sends the logs to the container orchestration API server. The container orchestration API server returns the logs to cluster user.

[0158] In accordance with an embodiment, when customer requests to delete a virtual agent, a safe delete is implemented before tearing down the virtual node replica. The safe delete includes multiple steps. The management plane updates virtual agent replicas with a flag to indicate DELETING status. The virtual agent leader replica cordons the virtual agent by sending a request to container orchestration API server. The virtual agent leader replica drains the virtual agent by evicting the pods scheduled on the virtual agent. The Management Plane tears down the virtual agent replicas. The Control Plane deletes the virtual node resource. A Cloud Controller Manager (CCM) deletes virtual agent resource from the cluster. Garbage collection takes care of releasing any container instances launched by a virtual agent.

7. MISCELLANEOUS; EXTENSIONS

[0159] Unless otherwise defined, terms (including technical and scientific terms) are to be given their ordinary and customary meaning to a person of ordinary skill in the art, and are not to be limited to a special or customized meaning unless expressly so defined herein.

[0160] This application may include references to certain trademarks. Although the use of trademarks is permissible in patent applications, the proprietary nature of the marks should be respected and every effort made to prevent their use in any manner which might adversely affect their validity as trademarks.

[0161] Embodiments are directed to a system with one or more devices that include a hardware processor and that are configured to perform any of the operations described herein and/or recited in any of the claims below.

[0162] In an embodiment, one or more non-transitory computer readable storage media comprises instructions which, when executed by one or more hardware processors, cause performance of any of the operations described herein and/or recited in any of the claims.

[0163] In an embodiment, a method comprises operations described herein and/or recited in any of the claims, the method being executed by at least one device including a hardware processor.

[0164] Any combination of the features and functionalities described herein may be used in accordance with one or more embodiments. In the foregoing specification, embodiments have been described with reference to numerous specific details that may vary from implementation to implementation. The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense. The sole and exclusive indicator of the scope of the disclosure, and what is intended by the applicants to be the scope of the disclosure, is the literal and equivalent

scope of the set of claims that issue from this application, in the specific form in which such claims issue, including any subsequent correction.

Claims

1. One or more non-transitory computer readable media comprising instructions that, when executed by one or more hardware processors, cause performance of operations comprising: executing a virtual agent in a first service tenancy of a cloud network manager, the virtual agent being executed on a virtual node of a container orchestration system; executing a container orchestration API server in a second service tenancy of the cloud network manager; and routing a request from the container orchestration API server to the virtual agent through a customer tenancy of a customer of the cloud network manager.
2. The non-transitory media of claim 1, wherein, the second service tenancy and the customer tenancy are implemented within a same cloud environment and configured to execute operations corresponding to a data set associated with the customer.
3. The non-transitory media of claim 1, wherein the first service tenancy, the second service tenancy and the customer tenancy are implemented within a same cloud environment and configured to execute operations corresponding to a data set associated with the customer.
4. The non-transitory media of claim 1, wherein routing the request through the customer tenancy comprises addressing the request to a service virtual network interface controller (VNIC) in the customer tenancy that sends the request to the virtual agent.
5. The non-transitory media of claim 1, wherein the virtual agent includes a first virtual agent replica executed by a first host and a second virtual agent replica executed by a second host, wherein the second virtual agent replica operates even when the first virtual agent replica fails.
6. The non-transitory media of claim 1, wherein the virtual agent interacts with containers that are executing at different locations, within the virtual node, than the virtual agent.
7. The non-transitory media of claim 1, wherein the request is to create, update or destroy containers for the virtual node, and wherein the container orchestration API server sends the request through the customer tenancy and wherein the request is rerouted to the virtual agent in the first service tenancy.
8. The non-transitory media of claim 1, wherein the virtual agent instructs a container instance control plane to create, update, or destroy container instances containing container orchestration pods.
9. The non-transitory media of claim 7, wherein the virtual agent, the container orchestration API server and container orchestration pods in container instances are part of a Kubernetes cluster and the container instances contain a kube-proxy.
10. The non-transitory media of claim 1, wherein the customer tenancy includes a customer network, and the virtual agent is attached to the customer network with a service virtual network interface controller (VNIC) with a single address for a first virtual agent replica and a second virtual agent replica executing on different hosts.
11. The non-transitory media of claim 10, wherein messages are load balanced by the Service VNIC between the first virtual agent replica and the second virtual agent replica.
12. The non-transitory media of claim 1, further comprising patching a version of the virtual agent in the first service tenancy under control of the cloud network manager without requiring a request from the customer.
13. The non-transitory media of claim 1, wherein the virtual agent in the first service tenancy is protected using cloud network security for the first service tenancy.
14. A method comprising: executing a virtual agent in a first service tenancy of a cloud network manager, the virtual agent being executed on a virtual node of a container orchestration system; executing a container orchestration API server in a second service tenancy of the cloud network

manager; and routing a request from the container orchestration API server to the virtual agent through a customer tenancy of a customer of the cloud network manager.

15. The method of claim 14, wherein, the second service tenancy and the customer tenancy are implemented within a same cloud environment and configured to execute operations corresponding to a data set associated with the customer.

16. The method of claim 14, wherein the first service tenancy, the second service tenancy and the customer tenancy are implemented within a same cloud environment and configured to execute operations corresponding to a data set associated with the customer.

17. The method of claim 14, wherein routing the request through the customer tenancy comprises addressing the request to a service virtual network interface controller (VNIC) in the customer tenancy that sends the request to the virtual agent.

18. The method of claim 14, wherein the virtual agent includes a first virtual agent replica executed by a first host and a second virtual agent replica executed by a second host, wherein the second virtual agent replica operates even when the first virtual agent replica fails.

19. The method of claim 14, wherein the virtual agent interacts with containers that are executing at different locations, within the virtual node, than the virtual agent.

20. A system comprising: at least one device including a hardware processor; the system being configured to perform operations comprising: executing a virtual agent in a first service tenancy of a cloud network manager, the virtual agent being executed on a virtual node of a container orchestration system; executing a container orchestration API server in a second service tenancy of the cloud network manager; and routing a request from the container orchestration API server to the virtual agent through a customer tenancy of a customer of the cloud network manager.
